B.TECH CCE REVISION · PART 4

Databases and Data Systems

Covers the marksheet subjects Database Systems and Database Systems Lab, and the storage side of Data Mining.

The relational model and SQL, schema design, indexes, transactions and isolation, SQL vs NoSQL vs vector stores, caching, replication and sharding, and warehouses vs operational databases.

Databases were rated "comfortable, want to deepen". So this chapter moves fast over the basics and spends its time on what decides whether a real product works: schema design, indexes, transactions, and choosing the right kind of store. Most products, Aerospeak included, are mainly a database with a user interface on top; most performance problems and many data-loss bugs live here.

> **How to read it.** The labs use SQLite, which is built into Python, so there is nothing to install. Do Lab 1 (schema) before Section 4 and Lab 2 (indexes) right after it.

Isagi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

Contents

1. What a database gives you that files don't
2. The relational model and SQL
3. Schema design and normalisation
4. Indexes and query plans
5. Transactions and ACID
6. Isolation levels and the anomalies they prevent
7. How a database stores data
8. SQL, NoSQL, and vector stores
9. Caching
10. Replication and sharding
11. Operational databases vs warehouses; event and analytics data
12. Labs
13. Self-check questions and answers
14. Resources

How to use this chapter

1. What a database gives you that files don't

A database is a program that stores data so that many users can read and change it at once, quickly, without losing or corrupting it.

You could keep everything in a JSON file. It breaks the moment you need any of these, which every real product does:

Need	What goes wrong with files	What a database does
Find one record among millions	Read the whole file	Indexes jump straight to it
Two users writing at once	One overwrites the other	Concurrency control and locking
Crash mid-write	Half-written, corrupt file	Transactions and a write-ahead log
Ask new questions	Write new code each time	A query language (SQL)
Keep data valid	Nothing stops bad data	Types, constraints, foreign keys
Control who sees what	All or nothing	Users, roles, permissions

The default choice for a new product is a relational database, usually **PostgreSQL** (or a hosted version such as Supabase, Neon, RDS). Choose something else only when you can name the reason.

2. The relational model and SQL

Data lives in tables of rows and columns; tables point at each other by keys; SQL describes what you want, and the database works out how to get it.

- A **table** (relation) has typed **columns**; each **row** is one record.
- A **primary key** uniquely identifies a row (`id`). A **foreign key** is a column that refers to another table's primary key (`sessions.user_id → users.id`), and the database can refuse rows that point at nothing.
- **SQL is declarative**: you say *what* you want; the **query planner** chooses *how* (which index, which join order). That's why the same query can be fast or slow depending on indexes and statistics.

```
-- Average score per user over the last 30 days, best first
SELECT u.name, COUNT(DISTINCT s.id) AS sessions, ROUND(AVG(sc.score), 1) AS avg_score
FROM users u
JOIN sessions s ON s.user_id = u.id
JOIN scores sc ON sc.session_id = s.id
WHERE s.started_at >= DATE('now', '-30 day')
GROUP BY u.id, u.name
HAVING COUNT(DISTINCT s.id) >= 3
ORDER BY avg_score DESC
LIMIT 10;
```

Reading order that makes SQL click: **FROM/JOIN** (which rows) → **WHERE** (filter rows) → **GROUP BY** (bucket them) → **HAVING** (filter buckets) → **SELECT** (compute columns) → **ORDER BY / LIMIT**.

Join	Returns
INNER JOIN	Only rows with a match on both sides
LEFT JOIN	Every row from the left table, with NULLs where the right has no match ("users, including those with no sessions")
FULL OUTER JOIN	Everything from both sides
CROSS JOIN	Every combination (rarely intended)

NULL means "unknown", not zero or empty. `NULL = NULL` is not true; use `IS NULL`. Many reporting bugs are NULLs silently dropped by a filter or an inner join.

3. Schema design and normalisation

Store each fact once, in the right place. Then denormalise deliberately, only where reads demand it.

Normalisation removes duplication so that data can't contradict itself:

- **1NF**: one value per cell. No comma-separated lists in a column.
- **2NF / 3NF**: every column depends on the key, the whole key, and nothing but the key. If a `sessions` table stores the user's college name, renaming the college means updating thousands of rows, and missing one leaves the data inconsistent. Store the college once, on the user (or a `colleges` table), and join.

Denormalisation copies data on purpose to make reads cheap: storing `avg_score` on the user row, or a pre-joined reporting table. The cost is keeping copies in sync. It's a reasonable trade for read-heavy dashboards; make sure one place is the source of truth and the rest are derived from it.

Worked example: a schema for a speaking-practice product

```
CREATE TABLE users (
  id          INTEGER PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE,
  name        TEXT NOT NULL,
  college     TEXT,
  created_at  TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE sessions (
  id          INTEGER PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id),
  kind        TEXT NOT NULL CHECK (kind IN ('interview','debate','negotiation')),
  started_at  TEXT NOT NULL,
  duration_s  INTEGER,
  audio_url   TEXT,                -- the file lives in object storage, not the DB
  transcript  TEXT
);
CREATE TABLE scores (
  session_id  INTEGER NOT NULL REFERENCES sessions(id),
  rubric      TEXT NOT NULL,       -- 'fluency', 'structure', 'confidence'...
  score       REAL NOT NULL CHECK (score BETWEEN 0 AND 10),
  scorer      TEXT NOT NULL,       -- 'llm:model-x:v3' or 'human:coach_12'
  PRIMARY KEY (session_id, rubric, scorer)
);
```

Design choices worth noticing:

- **Big blobs (audio) go in object storage** (S3, R2, Supabase Storage); the database keeps a URL. Databases are bad at multi-megabyte files.
- **Scores are rows, not columns.** Adding a new rubric doesn't need a schema change, and you can compare scorers.
- **Record who or what produced a score** (`scorer` with a model and prompt version). When you change the prompt, old and new scores stay distinguishable.
- **Constraints** (`NOT NULL`, `CHECK`, `UNIQUE`, foreign keys) are free bug-catchers; the database rejects nonsense at the door.
- Times are stored in UTC; convert to IST only for display.

4. Indexes and query plans

An index is a sorted lookup structure on one or more columns that lets the database find rows without scanning the whole table, at the cost of extra storage and slower writes.

- Without an index, `WHERE user_id = 42` reads every row: a **full table scan**. Fine at 1,000 rows, terrible at 10 million.
- Most indexes are **B-trees**: sorted, balanced trees a few levels deep. Finding one row among a billion takes ~4–5 page reads. They also serve range queries (`started_at > ...`) and `ORDER BY` for free.
- **Composite indexes** on `(user_id, started_at)` serve "this user's sessions, newest first" perfectly. Column order matters: the index helps queries that filter on the leftmost columns. It's like a phone book sorted by surname then first name: great for "all the Sharmas", useless for "everyone called Priya".
- **Primary keys and UNIQUE constraints** create indexes automatically. **Foreign keys usually don't** (in Postgres), and unindexed foreign keys are a classic cause of slow joins and deletes.
- **Costs**: every insert or update must also update every index. Ten indexes on a write-heavy table hurt.
- Indexes don't help if the query can't use them: wrapping the column in a function (`WHERE LOWER(email) = ...`), leading wildcards (`LIKE '%gmail'`), or filtering on a column that matches half the table anyway.

EXPLAIN shows the plan the database chose. `EXPLAIN ANALYZE` (Postgres) actually runs it and shows real times. The workflow for any slow query: run EXPLAIN → look for full scans on big tables → add the index that matches the WHERE and ORDER BY → run EXPLAIN again → confirm.

The N+1 problem: code loads 50 sessions, then loops and runs one query per session to fetch its scores: 51 queries and 51 round trips. Fix it with a join or a single `WHERE session_id IN (...)`. ORMs make this mistake easy to write; it's the most common cause of slow pages.

5. Transactions and ACID

A transaction groups several changes into one all-or-nothing unit.

Booking a coaching slot means: check the slot is free, mark it taken, charge credits, write a booking. If the server dies after charging but before booking, the user has paid for nothing. Wrapped in a transaction, either all four happen or none do.

Letter	Guarantee	Plain meaning
Atomicity	All or nothing	A crash mid-way leaves no half-done changes
Consistency	Rules hold	Constraints (foreign keys, CHECKs) are never violated by a committed transaction
Isolation	Concurrent transactions don't trip over each other	Each behaves (roughly) as if it ran alone; *how* roughly is the isolation level
Durability	Committed means saved	Once COMMIT returns, the data survives a crash (written to the log and fsynced)

```
BEGIN;
UPDATE slots   SET booked_by = 42 WHERE id = 7 AND booked_by IS NULL;
-- the app checks that exactly 1 row changed; if 0, someone else got it: ROLLBACK
UPDATE wallets SET credits = credits - 1 WHERE user_id = 42 AND credits >= 1;
INSERT INTO bookings (slot_id, user_id) VALUES (7, 42);
COMMIT;
```

The `WHERE ... AND booked_by IS NULL` is the database-level fix for the race condition from Part 3: the check and the change happen in one atomic statement.

Durability works through a **write-ahead log (WAL)**: every change is appended to a sequential log and flushed to disk *before* it's applied to the main data files. After a crash, the database replays the log. The same log feeds replication (Section 10).

6. Isolation levels

Full isolation is expensive, so databases offer levels that allow some anomalies in exchange for speed.

Anomaly	What happens
Dirty read	You read another transaction's uncommitted change, which then rolls back
Non-repeatable read	You read a row twice in one transaction and get different values
Phantom	You run the same query twice and new rows have appeared
Lost update	Two transactions read, modify, and write the same row; one change vanishes
Write skew	Two transactions each check a condition, both see it OK, and together they break it (two doctors both go off-call because each saw the other on call)

Level	Prevents	Notes
Read Uncommitted	Almost nothing	Rarely used
Read Committed	Dirty reads	**Postgres default**. Lost updates and write skew are still possible
Repeatable Read / Snapshot	+ non-repeatable reads, most phantoms	Each transaction sees a consistent snapshot. MySQL InnoDB default
Serializable	Everything	As if transactions ran one at a time. Some transactions get aborted and must be retried

Postgres implements this with **MVCC** (multi-version concurrency control): writers create new versions of rows instead of overwriting, so readers see a consistent snapshot without blocking writers. Old versions are cleaned up later by **VACUUM**.

The practical rules: use atomic conditional updates (`SET x = x - 1 WHERE x >= 1`) or `SELECT ... FOR UPDATE` for read-modify-write; use Serializable for money-like invariants and retry on failure; and test concurrency, because these bugs never show up with one user clicking.

7. How a database stores data

Under SQL, a database is pages on disk, a buffer of hot pages in memory, and a log.

- Data is stored in fixed-size **pages** (8 KB in Postgres). The **buffer pool** keeps hot pages in RAM. A query that hits only cached pages is fast; one that has to read many pages from disk is slow. "Add RAM" often fixes a database because more of the working set fits.
- **Row stores** (Postgres, MySQL) keep each row together: ideal for "fetch this user's record" and small writes (**OLTP**, online transaction processing).
- **Column stores** (BigQuery, Snowflake, ClickHouse, DuckDB) keep each column together: ideal for "average score across 50 million sessions", which reads two columns out of twenty and compresses beautifully (**OLAP**, analytics).
- **LSM trees** (Cassandra, RocksDB, many key-value stores) buffer writes in memory and flush sorted files, merging them in the background. Very fast writes, somewhat slower reads, compared to B-trees.
- **Connection limits**: each Postgres connection is a process with its own memory, so a database may handle only a few hundred. Serverless apps that open a connection per request exhaust this quickly; that's why **connection poolers** (PgBouncer, Supabase's pooler) exist.

8. SQL, NoSQL, and vector stores

Different stores optimise for different access patterns. Start relational; add a specialised store only for a specific need.

Type	Examples	Good at	Weak at
Relational	Postgres, MySQL, SQLite	Structured data, joins, transactions, ad-hoc queries	Massive write scale across many machines (needs effort)
Document	MongoDB, Firestore	Nested JSON per record, flexible schema, simple scaling	Joins across collections, multi-document transactions (improved but limited)
Key-value	Redis, DynamoDB	Lookups by key at huge scale, very low latency	Querying by anything other than the key
Wide-column	Cassandra, Bigtable	Enormous write volumes, time-series	Ad-hoc queries; you design tables per query
Search	Elasticsearch, OpenSearch, Typesense	Full-text search, relevance ranking, filters	Being the source of truth
Graph	Neo4j	Relationship-heavy queries ("friends of friends")	General workloads
Vector	pgvector, Pinecone, Qdrant, Weaviate	Nearest-neighbour search over embeddings (RAG, "find similar answers")	Exact filtering and transactions (varies)
Warehouse	BigQuery, Snowflake, ClickHouse	Analytics over billions of rows	Many small writes, low-latency lookups

"NoSQL" is mostly about trade-offs, not modernity. Document and key-value stores give up joins and some guarantees to scale out easily and keep a flexible shape. Postgres now handles JSON columns (`jsonb`) and vectors (`pgvector`) well enough that many products never need a second database.

Vector stores: an embedding model turns text (or audio) into a vector of ~1,000 numbers such that similar meanings land near each other. A vector index (usually **HNSW**, a navigable graph) finds the nearest vectors approximately but fast. That's the retrieval step in RAG. The questions to ask: how many vectors, do you need metadata filters ("only this user's documents"), and how fresh must it be. Below a few million vectors, `pgvector` inside your existing Postgres is usually the simplest choice.

9. Caching

Keep a copy of expensive results somewhere faster, and accept that the copy can go stale.

- **Where**: in the app process's memory, in a shared cache like **Redis**, in a CDN (Part 2), or in the browser.
- **Cache-aside** (the common pattern): look in the cache; on a miss, read the database and store the result with a **TTL** (expiry). Simple and robust.
- **Write-through**: write to the cache and the database together. Fresher, more complex.
- **Invalidation** is the hard part: when data changes, every cached copy is wrong until it expires or is deleted. Pick TTLs by how stale each thing may be (a leaderboard: a minute is fine; a wallet balance: never cache it naively).
- **Cache stampede**: a popular key expires and a thousand requests hit the database at once. Fixes: stagger TTLs, let one request recompute while others wait or get the stale value.
- **LLM caching**: identical prompts can return cached answers, and providers offer *prompt caching* that makes a long, repeated prefix (instructions, documents) cheaper and faster. Check your provider's current terms.

10. Replication and sharding

Replication copies the same data to several machines (for reads and survival). Sharding splits different data across machines (for size and write load).

Replication

- A **primary** takes writes and streams its log to **replicas**, which serve reads and can take over if the primary dies (**failover**).
- **Synchronous** replication waits for a replica to confirm each write: no data lost on failover, slower writes. **Asynchronous** doesn't wait: faster, but the last few seconds of writes can be lost, and replicas lag behind.
- **Replication lag** causes "read your own write" bugs: a user updates their profile (written to the primary), the next page reads a replica, and the old value appears. Fixes: read from the primary right after a user's own write, or use sticky routing.

Sharding (partitioning)

- Split rows across machines by a **shard key**: by `user_id` hash, by region, by tenant. Each shard is a smaller database.
- Choosing the key is the whole game: queries that include it go to one shard (fast); queries that don't must ask every shard (slow). Cross-shard transactions and joins become hard.
- **Hot shards**: if one key is much busier (one huge customer), its shard overloads.
- **Don't shard early.** A single well-indexed Postgres on a large machine handles far more than most startups ever need, with read replicas for scale. Sharding is a cost you take on when you've run out of options.

The trade-offs between consistency and availability across machines are covered in Part 7 (CAP and friends).

11. Operational databases vs warehouses; event and analytics data

The database that runs the product and the one that answers business questions should usually be different.

	Operational (OLTP)	Warehouse (OLAP)
Serves	The app: sign-ups, sessions, payments	Analysts and dashboards
Queries	Small, frequent, by key	Large scans and aggregations
Storage	Row store	Column store
Freshness	Live	Minutes to a day behind
Examples	Postgres, MySQL	BigQuery, Snowflake, ClickHouse, DuckDB

Data moves from operational systems to the warehouse through **ETL/ELT** pipelines (extract, load, transform, with tools like Airbyte, Fivetran and dbt) or by streaming changes (**CDC**, change data capture). Running heavy analytics directly on the production database slows the product down for users.

Event data

Product analytics is built on **events**: immutable records of something that happened.

```
event_id | user_id | event_name        | occurred_at (UTC)    | properties (JSON)
---------+---------+-------------------+----------------------+------------------------------
--
91a7…    | 42      | session_started   | 2026-11-12T04:10:03Z |
{"kind":"interview","device":"android"}
91a8…    | 42      | answer_submitted  | 2026-11-12T04:12:41Z |
{"question_id":7,"duration_s":95}
91a9…    | 42      | feedback_viewed   | 2026-11-12T04:13:05Z | {"rubrics":
["fluency","structure"]}
```

- Events are **append-only**: never update them; corrections are new events.
- Keep a **tracking plan**: a list of event names and their properties, agreed in advance. Inconsistent names (`SessionStart`, `session_started`, `start_session`) ruin analysis.
- Events answer funnel and retention questions ("what share of users who start a session view the feedback? who comes back a week later?") that the operational tables answer badly.
- In a warehouse the classic layout is a **star schema**: a central **fact** table of events or measurements surrounded by **dimension** tables (users, colleges, questions, dates).

12. Labs

All labs use Python's built-in `sqlite3`.

Lab 1. Build the schema. Create the three tables from Section 3 in a file `practice.db`. Insert 3 users, 10 sessions, and a few scores. Try to insert a score of 12 and a session for user 999; see which constraints catch what. (In SQLite, run `PRAGMA foreign_keys = ON;` first; it's off by default.)

Lab 2. Watch an index change a query plan.

```
import sqlite3, random, time
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE sessions (id INTEGER PRIMARY KEY, user_id INT, started_at TEXT, score
REAL)")
db.executemany("INSERT INTO sessions (user_id, started_at, score) VALUES (?,?,?)",
    [(random.randint(1, 50_000), f"2026-{random.randint(1,12):02d}-
{random.randint(1,28):02d}", random.random()*10)
     for _ in range(1_000_000)])
q = "SELECT * FROM sessions WHERE user_id = 4242 ORDER BY started_at DESC"
def run():
    plan = db.execute("EXPLAIN QUERY PLAN " + q).fetchall()
    t = time.perf_counter(); db.execute(q).fetchall()
    print([p[-1] for p in plan], f"{(time.perf_counter()-t)*1000:.1f} ms")
run()
db.execute("CREATE INDEX idx_user_time ON sessions(user_id, started_at)")
run()
```

The plan changes from a full scan with a temporary sort to a search using the index, and the time drops by orders of magnitude.

Lab 3. Break and fix a transaction. Write a script that moves credits between two wallets with two UPDATEs. Raise an exception between them and check the balances: money vanished. Now wrap them in `with db:` (a transaction in Python's sqlite3) and repeat: the first update is rolled back.

Lab 4. Write five analytics queries on your Lab 1 data: sessions per user, average score per rubric, users with no sessions (LEFT JOIN + IS NULL), each user's best session, and week-over-week session counts.

Lab 5. Design events. Write a tracking plan of 8–12 events for a product you're building, with properties for each. For each event, write the question it lets you answer.

13. Self-check questions

1. **Why not store everything in JSON files?** No fast lookups, no safe concurrent writes, corruption on crash, no constraints, no ad-hoc queries.
2. **What does normalisation buy you, and when do you break it?** Each fact stored once, so data can't contradict itself. Break it deliberately for read-heavy paths, with one clear source of truth.
3. **What is an index, in one sentence?** A sorted structure on some columns that lets the database find matching rows without scanning the table, at the cost of storage and slower writes.
4. **Why might an index on (user_id, started_at) not help a query on started_at alone?** The index is sorted by user_id first; without a user_id filter the database can't jump to a range.
5. **What's the N+1 problem?** One query for a list, then one query per item: many round trips. Fix with a join or a single IN query.
6. **Explain ACID to a non-technical person.** All steps happen or none do; the rules always hold; simultaneous users don't corrupt each other's work; once saved, it stays saved.
7. **How do you stop two users booking the last slot?** Make the check and update one atomic statement (UPDATE ... WHERE booked_by IS NULL, then check rows affected), or lock the row, or use Serializable and retry.
8. **What's write skew?** Two transactions each check a condition on overlapping data, both see it satisfied, both write, and together they break it. Read Committed doesn't prevent it.
9. **Row store vs column store?** Row stores suit fetching and writing whole records (apps); column stores suit scanning a few columns across huge numbers of rows (analytics).
10. **When would you add a vector store?** For similarity search over embeddings, such as retrieval in RAG. At modest scale, pgvector in the existing Postgres is usually enough.
11. **What's the hard part of caching?** Invalidation: knowing when a cached copy is stale and removing or refreshing it.
12. **Replication vs sharding?** Replication copies all data to more machines (read scale, survival). Sharding splits data across machines (size and write scale), at the cost of cross-shard queries.
13. **Why can a user not see their own update right after saving?** The read hit an asynchronous replica that hasn't caught up (replication lag).
14. **Why keep analytics off the production database?** Big scans compete with user traffic and slow the product; warehouses are built for that work.

14. Resources

Check these are still available before committing time.

- **Martin Kleppmann, *Designing Data-Intensive Applications*.** The single best book for this chapter and Part 7. Chapters 2, 3, 5, 6, 7 cover data models, storage, replication, partitioning and transactions. A second edition has been in progress; check which is current.
- **CMU 15-445 Intro to Database Systems** lectures (Andy Pavlo, free on YouTube). Rigorous and entertaining; watch storage, indexes, query execution and concurrency control.
- **Markus Winand, *Use the Index, Luke*** (free, use-the-index-luke.com). Everything practical about indexes.
- **PostgreSQL documentation**, especially the chapters on indexes, EXPLAIN and concurrency control. Unusually readable.
- **SQLBolt** or **Mode's SQL tutorial** for quick interactive SQL practice.